

SmartLogix

Plan de Branching

Estrategia de ramas Git para el equipo de desarrollo

GitFlow

Feature Branching

Pull Requests

Semantic Versioning

Institución: Duoc UC

Fecha: Abril 2026

Versión: 1.0.0

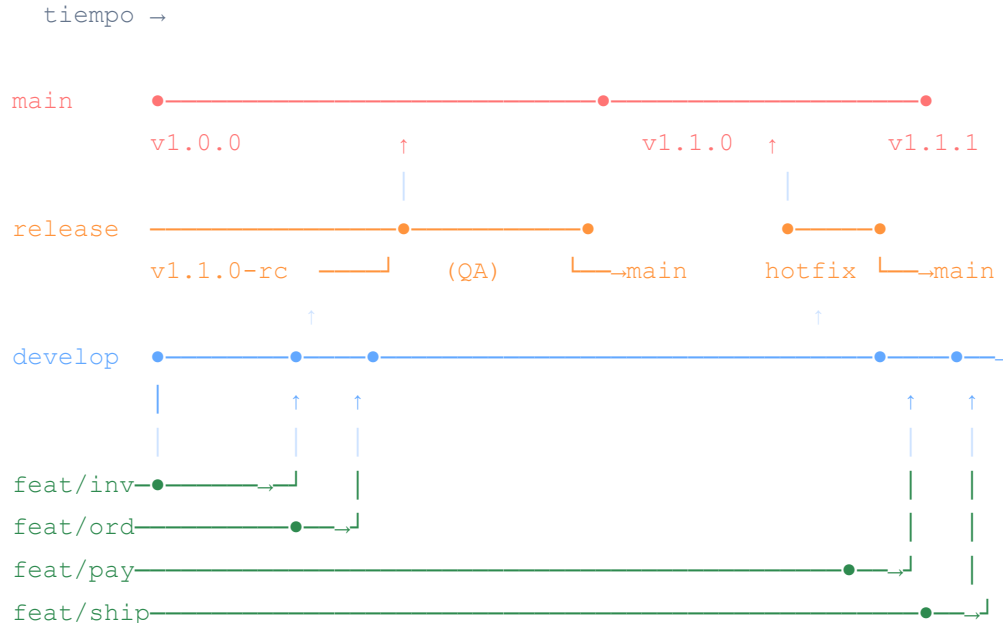
1. Introducción

Este documento describe la estrategia de branching adoptada por el equipo de SmartLogix para organizar el trabajo en paralelo, controlar las versiones del software y garantizar que el código en producción siempre sea estable.

La estrategia elegida es **GitFlow adaptado**, basada en el modelo propuesto por Vincent Driessen (2010), con ajustes para equipos pequeños que trabajan en sprints cortos. Combina ramas de larga duración (`main` , `develop`) con ramas de corta duración (`feature` , `release` , `hotfix`).

Principio guía: La rama `main` solo contiene código que ya fue probado y aprobado. Nadie hace push directo a `main` ni a `develop` .

2. Diagrama General de Ramas



Cada desarrollador trabaja en su propia rama `feature` . Al terminar, abre un Pull Request hacia `develop` . Cuando se acumula un conjunto de features probadas, se crea una rama `release` para QA final antes de fusionar con `main` .

3. Tipos de Ramas

main

Rama permanente — Producción

- Propósito:** Contiene el código que está o estuvo en producción. Cada commit representa una versión liberada.
- Protegida:** Sí. Solo acepta merges desde `release/*` y `hotfix/*` mediante PR aprobado.
- Etiquetas:** Cada merge lleva un tag de versión semántica: `v1.0.0`, `v1.1.0`, etc.
- Push directo:** Prohibido para todos los integrantes.

develop

Rama permanente — Integración

- Propósito:** Base de integración donde se acumulan las features terminadas. Representa el estado del próximo release.
- Protegida:** Sí. Solo acepta merges desde `feature/*` mediante PR con al menos 1 aprobación.
- Tests:** Se ejecutan automáticamente al recibir un PR (CI/CD).
- Push directo:** Prohibido.

feature/nombre-corto

Rama temporal — Desarrollo de funcionalidad

- Propósito:** Desarrollo de una nueva funcionalidad o mejora. Una rama por tarea o ticket.
- Origen:** Se crea desde `develop`.
- Destino:** Se fusiona hacia `develop` mediante PR.
- Duración:** Corta. Máximo la duración del sprint (1-2 semanas).
- Ejemplos:**
`feature/inventory-stock-alert`
`feature/orders-payment-confirm`
`feature/shipping-tracking-ui`

release/v1.x.0

Rama temporal — Preparación de versión

- Propósito:** Estabilizar el código antes de liberar. Solo se permiten bugfixes menores y ajustes de versión.
- Origen:** Se crea desde `develop`.
- Destino:** Se fusiona hacia `main` Y hacia `develop` (para sincronizar los fixes).
- Ejemplos:** `release/v1.1.0` , `release/v1.2.0`

hotfix/descripcion

Rama temporal — Corrección urgente en producción

- Propósito:** Corregir un bug crítico en producción sin esperar el próximo ciclo de release.
- Origen:** Se crea desde `main`.
- Destino:** Se fusiona hacia `main` Y hacia `develop`.
- Ejemplos:** `hotfix/gateway-502-dns` , `hotfix/payment-timeout`

4. Convención de Nombres

Tipo	Formato	Ejemplo
main	<code>main</code>	<code>main</code>
develop	<code>develop</code>	<code>develop</code>
feature	<code>feature/servicio-descripcion</code>	<code>feature/inventory-low-stock-alert</code>
release	<code>release/vMAJOR.MINOR.0</code>	<code>release/v1.2.0</code>
hotfix	<code>hotfix/descripcion-corta</code>	<code>hotfix/bff-cors-error</code>

Reglas de nomenclatura: Usar solo minúsculas, guiones medios como separador, sin espacios ni caracteres especiales. El nombre debe describir la funcionalidad, no el número de ticket.

5. Flujo de Trabajo Detallado

5.1 Desarrollar una nueva funcionalidad

1

**Crear
la
rama
desde
develop**

```
git
checkout
develop
&&
git
pull
origin
develop
git
checkout
-b
feature/inventory-
warehouse-
delete
```

2

**Desarrollar
y
hacer
commits
frecuentes**

```
git
add
src/controllers/inventoryController.js
git
commit
-m
"feat(inventory):
add
warehouse
delete
endpoint"
```

3

**Subir
la
rama
al
repositorio
remoto**

```
git  
push  
origin  
feature/inventory-  
warehouse-  
delete
```

4

**Abrir
Pull
Request
hacia
develop
en
GitHub**

Descripción
del
PR:
qué
hace,
cómo
probarlo,
capturas
si es
UI.
Asignar
al
menos
1
revisor
del
equipo.

**Revisión
de
código
y
aprobación**

El
revisor
aprueba
o
solicita
cambios.
Se
resuelven
los
comentarios.

**Merge
con
Squash
hacia
develop**

Se
usa
Squash
and
merge
para
mantener
el
historial
de
`develop`
limpio.
Se
elimina
la
rama
feature
después
del
merge.

5.2 Crear un Release

**Crear
rama
release
desde
develop**

```
git  
checkout  
-b  
release/v1.1.0  
develop
```


2

**Actualizar
versión
en
package.json
de
cada
servicio**

```
npm
version
minor
--
no-
git-
tag-
version
(en
cada
servicio)
```

3

**Pruebas
de
regresión
y
correcciones
menores**

Solo
bugfixes.
No
se
agregan
features
nuevas
en
esta
rama.

4

**Merge
hacia
main
y
etiqueta**

```
git
checkout
main
&&
git
merge
--
no-
ff
release/v1.1.0
git
tag
-a
v1.1.0
-m
"Release
v1.1.0"
```

5

**Sincronizar
develop
con
los
fixes
del
release**

```
git
checkout
develop
&&
git
merge
--
no-
ff
release/v1.1.0
```

6. Convención de Commits

Se utiliza **Conventional Commits** para mantener un historial legible y poder generar changelogs automáticos.

Prefijo	Cuándo usarlo	Ejemplo
feat:	Nueva funcionalidad	feat(orders): add payment confirmation flow
fix:	Corrección de bug	fix(gateway): resolve DNS cache on container restart
docs:	Solo documentación	docs(readme): add Docker setup instructions
refactor:	Refactorización sin cambio funcional	refactor(inventory): extract stock logic to repository
test:	Agregar o modificar tests	test(bff): add integration test for products endpoint
chore:	Tareas de mantenimiento	chore: update dependencies to latest minor
ci:	Cambios en CI/CD	ci: add SonarQube quality gate step

Formato completo: tipo(scope): descripción en imperativo, minúscula, sin punto final

El *scope* es opcional pero recomendado: indica el servicio o módulo afectado.

7. Reglas de Protección en GitHub

Regla	main	develop
Requiere Pull Request	✅ Sí	✅ Sí
Aprobaciones requeridas	2 aprobaciones	1 aprobación
Tests de CI deben pasar	✅ Sí	✅ Sí
Push directo permitido	❌ No	❌ No
Force push permitido	❌ No	❌ No
Eliminar rama permitido	❌ No	❌ No

8. Estructura de Repositorios

SmartLogix utiliza una estrategia de **multi-repositorio**: cada componente tiene su propio repositorio en GitHub, permitiendo que cada equipo despliegue de forma independiente.

Repositorio	Componente	Rama principal
smartlogix-frontend	React + Vite	main
smartlogix-bff	BFF Node.js	main
smartlogix-inventory	Microservicio inventario	main
smartlogix-orders	Microservicio pedidos	main
smartlogix-shipping	Microservicio envíos	main
smartlogix-payment	Microservicio pagos	main
smartlogix-archetypes	Arquetipos Maven	main

Repositorio principal: Existe un repositorio raíz `smartlogix` que contiene el `docker-compose.yml` y la documentación general del proyecto. Todos los repositorios de servicios son submódulos Git de este repositorio raíz.

9. Resumen de la Estrategia

Aspecto	Decisión	Justificación
Modelo base	GitFlow adaptado	Separa claramente desarrollo, QA y producción
Ramas permanentes	main + develop	develop como buffer entre features y producción
Ramas temporales	feature, release, hotfix	Ciclo de vida claro: se crean y eliminan
Integración	Pull Requests obligatorios	Revisión de código entre pares, trazabilidad
Merge strategy	Squash hacia develop, No-FF hacia main	Historial limpio en develop, trazable en main
Commits	Conventional Commits	Changelogs automáticos, historial legible
Versiones	Semantic Versioning (semver)	MAJOR.MINOR.PATCH con tags en main
Repositorios	Multi-repo	Despliegues independientes por servicio

Conclusión: La estrategia GitFlow adaptada permite al equipo de SmartLogix trabajar en paralelo sobre distintos microservicios sin interferencias, garantizando que `main` siempre refleje un sistema estable y listo para producción. La combinación de Pull Requests, Conventional Commits y Semantic Versioning otorga trazabilidad completa del historial de cambios.